JORNADA AUTOMAÇÃO NA VEIA

Estruturas Principais da Programação no Arduino

Este é um documento de apoio exclusivo para os alunos do curso Automação na Veia, sua reprodução ou distribuição sem autorização da CEMF Tecnologia é proibida.

Sumário

1- A ESTRUTURA DA LINGUAGEM DO ARDUINO....3
1.1 – Constantes e Variáveis....5
1.2 – Arrays....8
1.3 – Lógica e Aritmética....9
1.4 – Funções Matemáticas de Campos....10
1.5 – Funções para Controle de Fluxo....12
2 - AS PORTAS DE E/S DO ARDUINO E SUAS FUNÇÕES....19
2.1 – Os pinos digitais....19
2.2 – Os pino analógicos....24
2.4 – Os pinos de alimentação....28

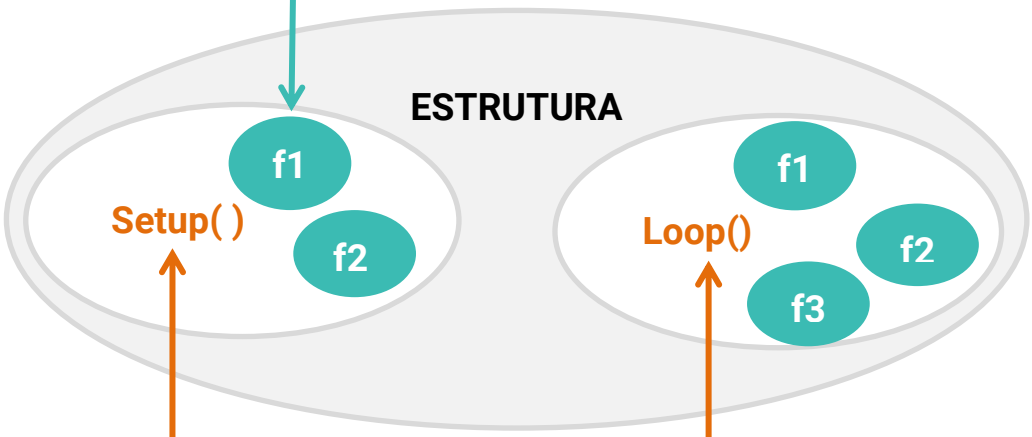
1- A ESTRUTURA DA LINGUAGEM DO ARDUINO

A estrutura básica da linguagem de programação do Arduino é bastante simples; ela é formada por dois blocos de funções que carregam outros blocos de funções escritas em linguagem C/C++. O primeiro bloco de funções forma a função **setup( );** o segundo, a função **loop().**

Funções em linguagens de programação são como sub-rotinas ou procedimentos; são pequenos blocos de programas usados para montar o programa principal.

Elas são escritas pelo programador para realizar tarefas repetitivas ou podem ser importadas prontas para o programa em forma de bibliotecas.

Declaração da Função toda função deve ser declarada antes de ser chamada atribuindo-lhe um tipo e um nome seguido de parênteses, onde serão colocados os parâmetros de passagem da função. Depois do nome são definidos entre as chaves '{' e '}' os procedimentos que a função irá executar.



Setup( ) Essa é a primeira função a ser chamada quando o programa inicia. E é executada apenas nessa primeira vez. Esta é uma função de preparação, ela dita o comportamento dos pinos do Arduino e inicializa a porta serial.

Loop( ) A função loop() é chamada logo a seguir e todas as funções embarcadas nela são repetidamente executadas. Ela fica lendo os pinos de entrada do Arduino e comandando os pinos de saída e a porta serial

Os Símbolos usados na construção de funções são os seguintes:

{ } → Dentro das **chaves** vão os procedimentos (statements) que a função deve executar;

; → O **ponto-e-vírgula** é usado para marcar o final de um procedimento;

// → Utilizado para criar um **comentário na linha:** qualquer caracter depois das duas barras é ignorado pelo programa;

/*...*/ → Utilizado para criar um bloco de comentários (várias linhas) : qualquer texto colocado entre esses símbolos também é ignorado pelo programa.

Exemplo:

```
/*
 *  Nesse código a função setup() prepara a porta serial para trabalhar
 *  com a velocidade de 9600 bits/s.
 *  Já a função loop() fica transmitindo a frase "Jornada AV - Seja Bem Vindo !!!"
 *  pela porta serial a cada 2 segundos.
*/

void setup() {
   Serial.begin(9600);                    // inicializando a portal serial
}

void loop() {
  Serial.println("Jornada AV - Seja Bem Vindo !!!");    // Imprimindo dados na Serial
  delay(2000);                            // Aguardando por 2 segundos
}
```

1.1 – Constantes e Variáveis

CONSTANTES são valores pré-definidos que nunca podem ser alterados. Na linguagem C do Arduino temos 3 grupos de constantes; os dois componentes de cada grupo sempre podem ser representados pelos números binários 1 e 0.

TRUE/FALSE são constantes booleanas que definem estados lógicos.

TRUE (VERDADEIRO) é qualquer valor que não seja zero.
FALSE (FALSO) é sempre zero.

HIGH/LOW essas constantes definem as tensões nos pinos do Arduino.

HIGH (ALTA) é uma tensão de 5 volts.
LOW (BAIXA) é o terra ou 0 volts.

INPUT/OUTPUT são constantes programadas pela função **pinMode()** para os pinos do Arduino.

INPUT (ENTRADA) entrada de sensores.
OUTPUT (SAÍDA) saída para controle.

VARIÁVEIS são posições na memória de programa do Arduino marcadas com um nome e o tipo de informação que deseja guardar. Essas posições podem receber um valor inicial. Os valores das variáveis podem ser alterados pelo programa.

Escopo da Variável é o limite ou abrangência da variável. Uma variável pode ser declarada em qualquer parte do programa. Se for declarada logo no início, **antes da função setup(),** ela tem o escopo de **Variável Global**, e por isso ela pode ser vista e usada por qualquer função no programa. Se declarada **dentro de uma função** ela tem escopo de **Variável Local**, e só pode ser usada por essa função.

Declaração da Variável como as funções, toda variável deve ser declarada antes de ser chamada. Essa declaração consiste em atribuir previamente um tipo e um nome a variável.

Tipos de Variáveis:

byte: esse tipo armazena 8 bits (0-255);

int: armazena números inteiros de até 16 bits;

long: armazena números inteiros de até 32 bits;

float: variáveis deste tipo podem armazenar números fracionários de até 32 bits.

Exemplo:

```
/*
 *  Esse programa aumenta e diminui gradativamente o brilho
 *  de um LED conectado no pino PWM 10 do Arduino
*/

int i = 0;                //declaração da variável global inteira 'i' iniciada com 0

void setup() {
   pinMode(10, OUTPUT);  // aqui 2 parâmentros são passados à função pinMode();
}

void loop() {
  for(i=0; i <= 255; i++) ledON();    // aumenta o brilho do led
  for(i=255; i >= 0; i--) ledON();    // diminui o brilho do led
}

void ledON(){             // função que acende o led
  analogWrite(10,i);      // o número do pino e o valor de i são passados à função analogWrite()
  delay(10);              // aguarda por 10 milissegundos
}
```

1.2 – Arrays

ARRAYS são coleções de variáveis do mesmo tipo, portanto são posições na memória de programa, com endereços que podem ser acessados por meio de um identificador, chamado de índice. A primeira posição de um array é sempre a de índice 0.

1) Declaração de um Array Os arrays, como as variáveis e as funções, devem ser declaradas com um tipo e um nome seguido de colchetes; e podem também ser inicializadas com os valores entre as chaves. Exemplo:

int nomeArray [ ] = {45,20,35,...};

2) Declaração de um Array Pode-se também declarar somente o tipo, o nome e o tamanho do array, deixando para o programa o armazenamento de variáveis nas posições, ou índices do array.

int nomeArray [5];

Escrever/Ler um Array Para guardar o inteiro 10 na posição 5 do array **nomeArray**, usa-se:

nomeArray [5] = 10;

Para atribuir o valor armazenado na 5ª posição de **nomeArray** à variável **x,** usa-se:

int x = nomeArray [5] ;

1.3 – Lógica e Aritmética

Operações Aritméticas e lógicas. As 4 operações aritméticas, **divisão, multiplicação, adição e subtração**, são representadas pelos símbolos: **/, *, + e -,** respectivamente, separando os operandos. E são 3 os operadores lógicos na linguagem do Arduino que são usados para comparar duas expressões e retornar a constante **TRUE/FALSE**

Símbolos compostos são aqueles que combinam os símbolos aritméticos entre si e com o sinal de atribuição:

```
x ++ ;          // x = x + 1
x -- ;          // x = x - 1
x += y ;        // x = x + y
x -= y ;        // x = x - y
x *= y ;        // x = x * y
x /= y ;        // x = x / y
```

Símbolos de comparação comparam uma variável com uma constante, ou variável entre si. São usadas para testar se uma condição é verdadeira.

```
x == y ;        // x é igual a y
x != y ;        // x é diferente de y
x < y ;         // x é menor que y
x > y ;         // x é maior que y
x <= y ;        // x é menor ou igual a y
x >= y ;        // x é maior ou igual a  y
```

Operadores lógicos são usados para comparar duas expressões, retornam 1 ou 0 (verdadeiro/falso).

&&	comparação 'E'
||	comparação 'OU'
!	lógica de negação 'NÃO'

1.4 – Funções Matemáticas de Campos

delay(ms) Essa função pausa o programa por um período em milissegundos indicado pelo parâmetro entre parênteses.

Exemplo:

```
delay(1000);
```

Com esse parâmetro o programa vai pausar durante 1 segundo (1000ms)

** Durante o período em que essa função está ativa qualquer outra função no programa é suspensa; Somente as interrupções de hardware podem parar essa função.*

delayMicroseconds(us) Essa função pausa o programa por um período em microssegundos indicado pelo parâmetro entre parênteses.

Exemplo:

```
delayMicroseconds(1000);
```

Com esse parâmetro o programa vai pausar durante 1 milissegundo(1000us)

** Durante o período em que essa função está ativa qualquer outra função no programa é suspensa; Somente as interrupções de hardware podem parar essa função.*

millis( ) Retorna o número de milissegundos desde que o Arduino começou a executar o programa corrente.

Exemplo:

unsigned long total = millis( );

Aqui a variável inteira longa desassinada (32 bits) chamada 'total' vai guardar o tempo em ms desde que o Arduino foi inicializado.

**Essa variável vai ser resetada depois de aproximadamente 50 dias quando chegar ao seu valor máximo 4.294.967.295, aí volta a zero e começa novamente.*

random(min, max) Gera números pseudo-aleatórios entre os limites **min** e **max** especificados como parâmetros.

Exemplo:

Int valor = random(20,600);

Á variável 'valor' vai ser atribuída um número inteiro qualquer entre 20 e 600.

O parâmetro **min é opcional e se excluído o limite mínimo é 0 por padrão.*

map(valor,min1, max1,min2,max2) A função **map( )** converte uma faixa de valores para outro faixa. O primeiro parâmetro 'valor' é a variável que será convertida; o segundo e o terceiro parâmetro são os valores mínimo e máximo dessa variável; o quarto e o quinto são os novos valores mínimo e máximo da variável 'valor'

Exemplo:

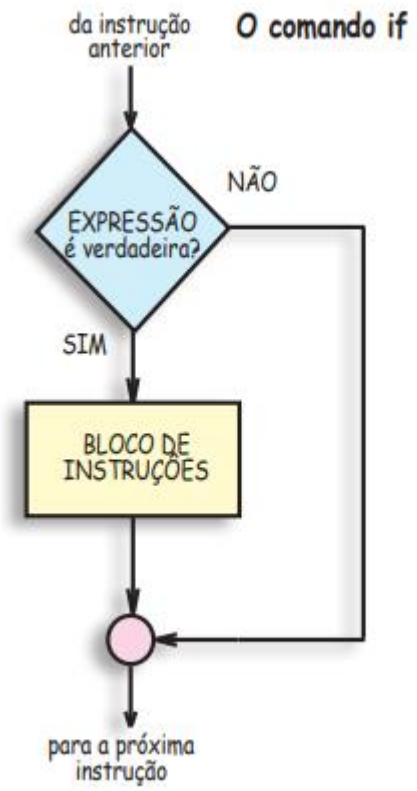
Int valor = map(analogRead(A0),0 1023,0,100);

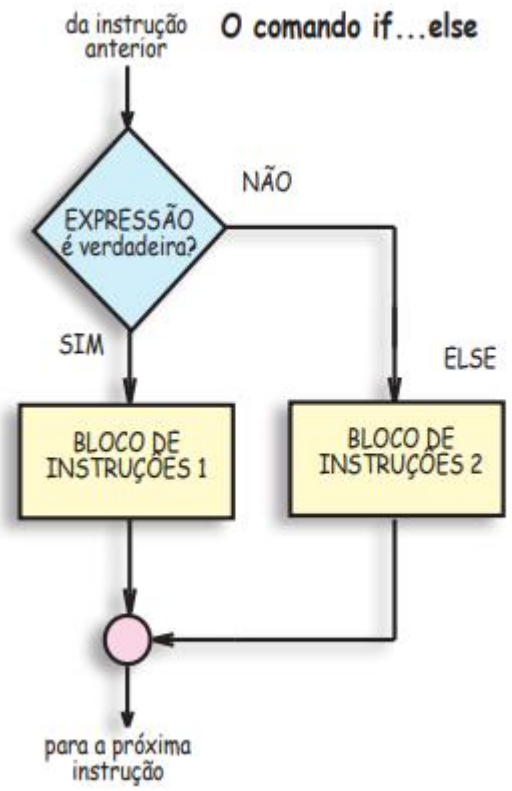
Á variável 'valor' vai guardar a leitura do nível analógico no pino A0 convertida da faixa 0-1023 para faixa 0-100

1.5 – Funções para Controle de Fluxo

if é um controle de fluxo usado para selecionar uma ou mais instruções baseadas no resultado de um teste de comparação. Todas as instruções entre as chaves **{** e **}** são executadas somente se o resultado deste teste for verdadeiro; se não, essas instruções não são executadas. Verdadeiro e qualquer resultado, mesmo negativo, diferente de zero. Falso é um resultado zero.

```
if(expressão){
        bloco de instruções;  // se 'expressão' for verdadeira, 'bloco de instruções' é executado
}
```



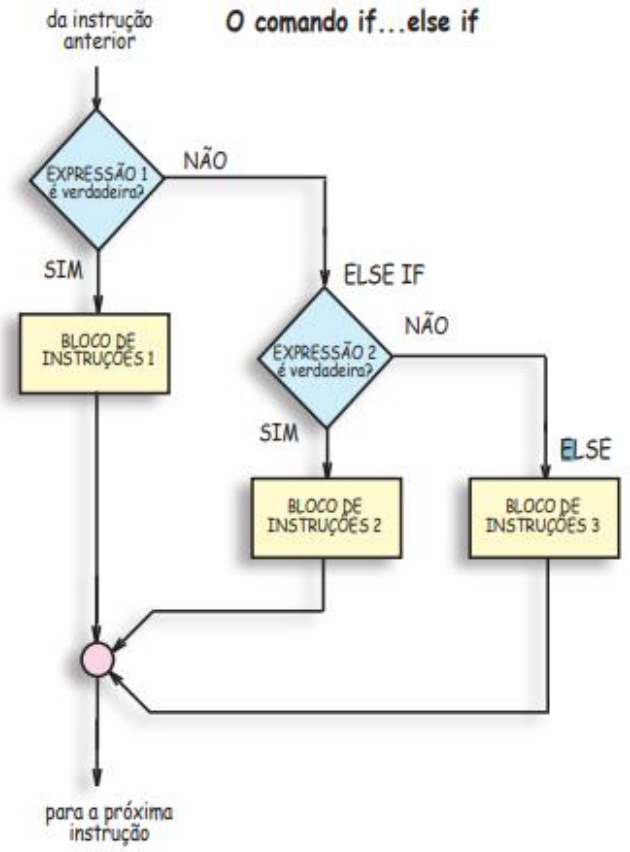


if...else Ao se acrescentar mais um bloco de instruções no loop do comando **if** pode-se criar o comando **if...else**, para fazer um teste novo quando o resultado da expressão for falsa.

```
If(expressão){
        bloco de instruções;  // se 'expressão' for verdadeira, 'bloco de instruções' é executado
}else{
        bloco de instruções2;  // se 'expressão' for falsa, 'bloco de instruções2' é executado
}
```

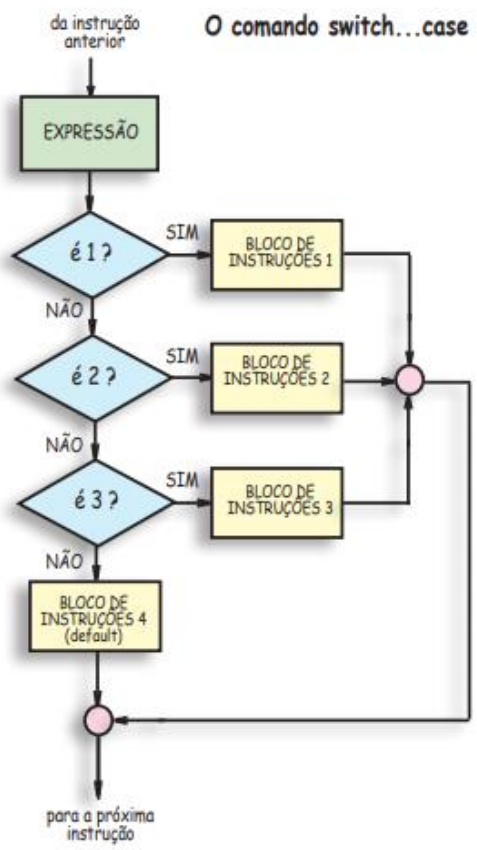
if...else if E de novo ao se acrescentar agora o comando **if..else** no loop do comando **if** pode-se criar mais um outro comando, o **if...else if**. No exemplo abaixo se **'expressão1'** for verdadeira o **'bloco de instrução1'** é executado; se **'expressão1'** for falsa mas **'expressão2'** for verdadeira **'bloco de instrução2'** é executado; e se **'expressão1'** e **'expressão2'** forem falsas o **'bloco de instrução3'** é executado.

```
If(expressão1){
        bloco de instruções1;
}
else If(expressão2){
        bloco de instruções2;
}
else{
        bloco de instruções3;
}
```



switch...case É possível ir inserindo comandos **if...else** na posição do segundo bloco de instruções de outro comando **if...else** e assim criar uma cadeia de comandos para testar dezenas de expressões até encontrar uma que retorne um resultado verdadeiro e executar um dos blocos de instruções; mas existe um comando próprio que simplifica bastante essa seleção, é o comando **switch...case**. Esse comando permite comparar uma mesma variável inteira, ou uma expressão que retorne um inteiro, com bários valores possíveis.

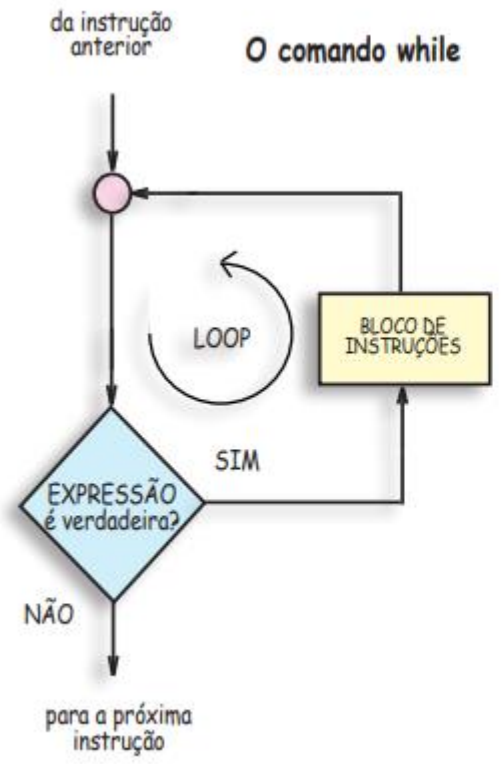
```
switch(expressão){
        case 1:         bloco de instruções1;
        break;
        case 2:         bloco de instruções2;
        break;
        case 3:         bloco de instruções3;
        break;
}
```

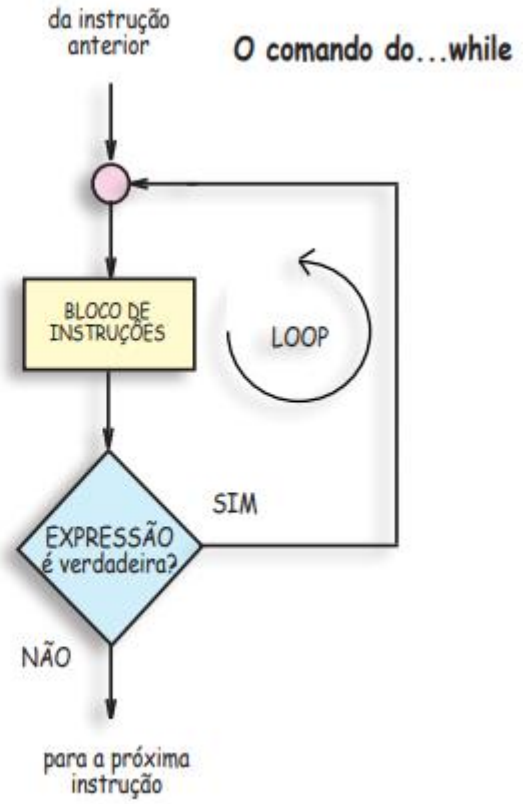


while Uma das operações mais frequentes que os programas executam é repetir um grupo de instruções até que uma condição inicialmente verdadeira se torne falsa. E para isso que serve o comando **while**. A sua sintaxe é a seguinte:

```
while(expressão){
        bloco de instruções;
}
```

O **bloco de instruções** será executado enquanto o parâmetro **expressão** for verdadeiro





do...while Para que o **'bloco de instruções'** seja executado apenas uma vez, ele é deslocado para a entrada da caixa de decisões, antes do teste de validade:

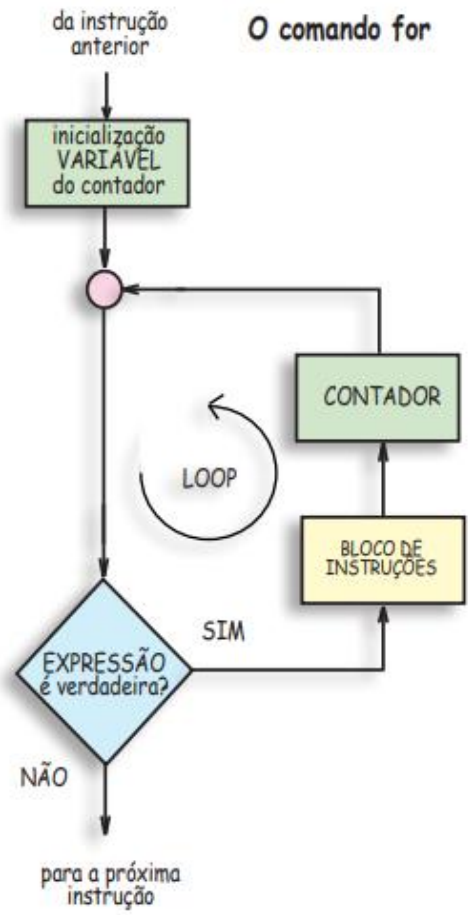
```
do{
        bloco de instruções;
}while(expressão);
```

Aqui o **'bloco de instruções'** será executado primeiro e só então o parâmetro **'expressão'** é avaliado.

for Inserindo-se no loop do comando **while** um contador que registre cada execução do bloco de instruções cria-se o comando for. Esse contador deve ter uma variável de controle que deve ser previamente inicializada com um tipo e um valor. A sua sintaxe é a seguinte:

```
for(variável; expressão; incremento){
        bloco de instruções;
}
```

A **variável** é inicializada normalmente com 0 ou 1; o parâmetro **expressão** deve conter o valor máximo (ou mínimo) que o contador deve alcançar; e **incremento** é o valor que será incrementado (ou decrementado) da variável cada vez que o bloco de instrução é executado. Observe que cada parâmetro entre parênteses é separado por ponto e vírgula.




```
for(int i = 0; i <= 10; i++){
  Serial.println(i);
  delay(1000);
}
```

Nesse exemplo a variável de controle 'i' do contador é inicializada com 0; o contador é testado e se o valor nele acumulado for menor ou igual a 10 seu valor é impresso

na serial, e depois de 1 segundo, o contador é incrementado e seu novo valor é testado novamente.

O operador ternário '?'

É possível simplificar códigos com comandos **if...else** em C/C++ com operador condicional '**?**', também chamado de operador ternário. Esse operador avalia uma expressão e se esta for verdadeira uma instrução é executada, se a expressão for falsa outra instrução é executada. A sua sintaxe é a seguinte

(expressão) ? instrução1 : instrução2;

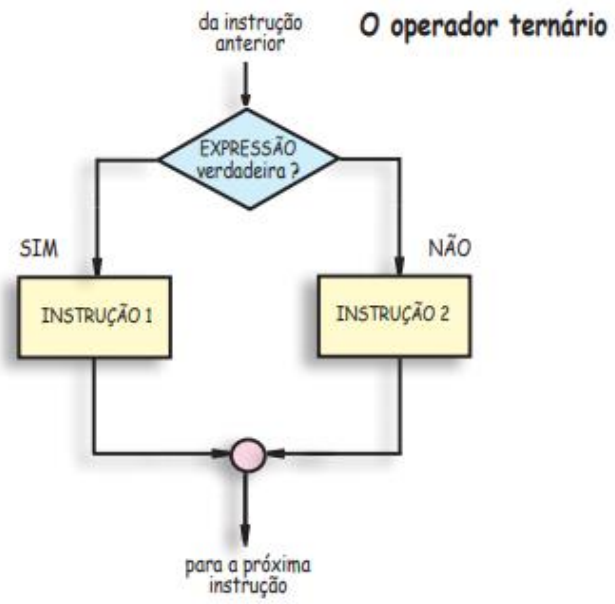
Note o uso e a posição entre as duas instruções de dois pontos na sintaxe deste operador

Exemplo:

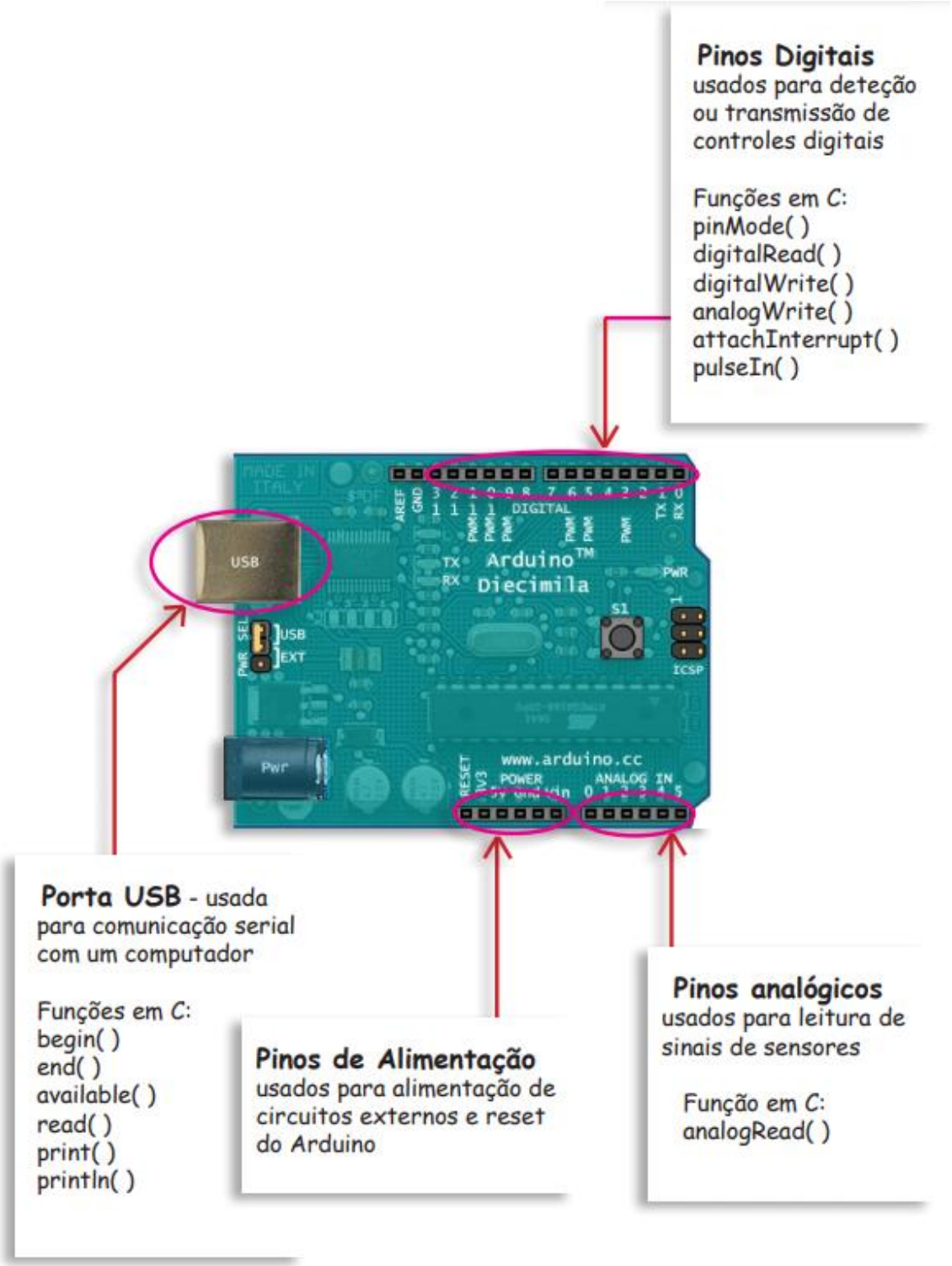
```
int x = 8;
y = (x < 10) ? 15: 20;
```

Aqui o valor de **y** vai depender da avaliação da expressão do operador ternário; como o valor de **x** vale 8, a expressão **(x < 10)** é verdadeira, por isso o inteiro 15 será atribuído a **y**; se o valor atribuído a **x** fosse maior que 10, **y** seria 20. Essa mesma expressão com o comando **if...else** ficaria assim:

```
int x = 8;
if(x < 10){
  y = 15;
}else{
  y = 20;
}
```



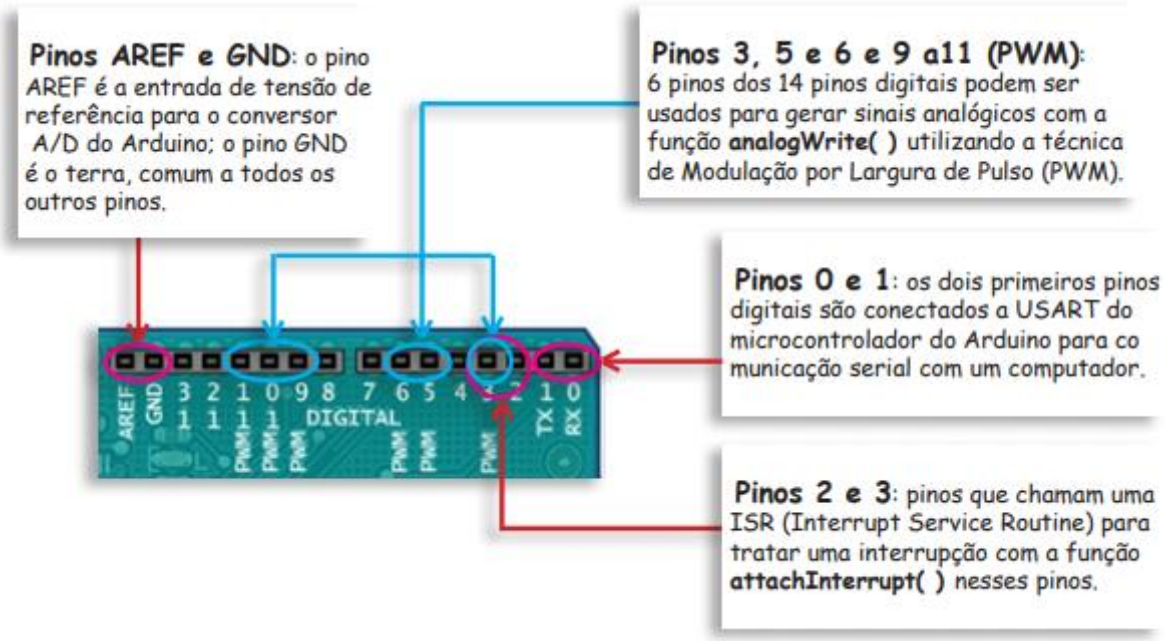
ENTRADAS E SAÍDAS DO ARDUINO



2 - AS PORTAS DE E/S DO ARDUINO E SUAS FUNÇÕES

2.1 – Os pinos digitais

Os **pinos digitais** são 14 pinos marcados com o nome **DIGITAL** logo abaixo, de duas barras de 8 pinos. São numerados de 0 a 13 da direita pra a esquerda e podem ser configurados pela função **pinMode()** para detectarem ou transmitirem níveis lógicos digitais (verdadeiro/falso, 1/0 ou HIGH/LOW).



Funções

pinMode(pino,modo)

Serve para estabelecer a direção do fluxo de informações em qualquer dos 14 pinos digitais. Dois parâmetros devem ser passados à função: o primeiro indica qual pino vai ser usado; o segundo, se esse pino vai ser entrada ou se vai ser saída dessas informações.

```
void setup() {

    pinMode(3,OUTPUT);
    /* Aqui o pino 3 é selecionado para transmitir
    informações do Arduino para um cirtuito externo
    qualquer. Para configurar esse pino como entrada,
    o segundo parâmetro dessa função deve ser INPUT*/

}
```

** Essa função é sempre escrita dentro da função setup*

digitalRead(pino)

Uma vez configurado um certo pino como entrada com a função **pinMode(),** a informação presente nesse pino pode ser lida com a função **digitalRead( )** e armazenada numa variável qualquer.

```
void loop() {

  int chave = digitalRead(3);
  /* Nesse exemplo a variável inteira 'chave' vai
  guardar o estado lógico (verdadeiro/falso) presente
  no pino digital 3 */

}
```

digitalWrite(pino,valor)

Para enviar um nível lógico para qualquer pino digital do Arduino utiliza-se essa função. Dois parâmetros são requeridos: o número do pino e o estado lógico (HIGH/LOW) em que esse pino deve permanecer.

```
void loop() {

  digitalWrite(3, HIGH);
  /* Aqui uma tensão de 5 volts é colocada no pino 3.
     Para enviar terra para esse pino o segundo
     parâmetro deverá ser LOW
  */

}
```

** É necessário configurar previamente o pino como saída com a função **pinMode()***

```
/*
 * No programa abaixo essas três funções são utilizadas para acender
 * um led no pino 3 todas vez que uma chave normalmente aberta no
 * pino 2 é pressionada.
*/
void setup() {
    pinMode(2,INPUT);          // chave no pino 2 como entrada
    pinMode(3,OUTPUT);         // led no pino 3 como saída
}

void loop() {
  int chave = digitalRead(2); // variável 'chave' guarda o estado do pino 2
  digitalWrite(3, chave);     // estado de 'chave' é passado para o pino 3
}
```

O código dentro da função **loop()** acima pode ainda ser simplificado da seguinte forma:

```
void loop() {
   digitalWrite(3,digitalRead(2));
   // led (pino 3) acende se a chave (pino 2) for pressionada
}
```

analogWrite(pino,valor)

O Arduino pode gerar tensões analógicas em 6 de seus 14 pinos digitais com a função **analogWrite( ).** Dois parâmetros devem ser passados à função: o primeiro indica em qual pino será gerada a tensão; o segundo determina a amplitude dessa tensão e deve ter valores entre 0 (para 0 volts) e 255 (para 5 volts).

```
void loop() {
    analogWrite(10, 128);
    /* Com esses parâmetros uma tensão analógica de 2,5 volts vai
     *  aparecer no pino 10. Não é necessário configurar um
     *  pino PWM como saída com a função pinMode() quando se chama
     *  a função analogWrite( ).
    */
}
```

** Modulação por Largura de pulsos, ou PWM (Pulse With Modulation) na língua inglesa, é uma técnica usada para gerar tensões analógicas a partir de uma sequência de pulsos digitais.*

attachInterrupt(pino, função, modo)

Essa função é uma rotina de serviço de interrupção, ou ISR (Interrupt Service Routine) em inglês. Toda vez que ocorrer uma interrupção por hardware no pino digital 2 ou 3 do Arduino uma outra função, criada pelo programador, vai ser chamada.

O terceiro parâmetro, **modo**, informa como a interrupção vai ser disparada, se na borda de subida do pulso detectado no pino do Arduino, se na borda de descida, se quando o pulso for baixo ou se na mudança de nível desse pulso.

```
void setup() {
    attachInterrupt(0, contador, RISING);
    /* Nesse exemplo a função 'contador' vai ser chamada quando o Arduino
    detectar uma mudança do nível LOW para o nível HIGH em seu pino 2.
    Nessa ISR o parâmetro 0 monitora o pino 2, o parâmetro 1 monitora
    o pino 3.*/
}
```

** LOW : dispara a interrupção quando pino está em 0;*

** CHANGE : dispara sempre que o pino muda de estado (de 0 para 1, ou vice versa);*

** RISING : somente quando o pino muda de 0 para 1*

** FALLING : somente quando o pino muda de 1 para 0.*

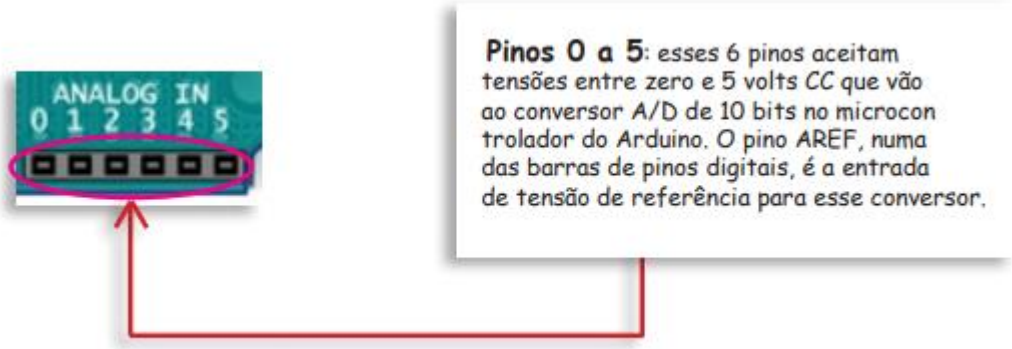
pulseIn(pino, valor, espera)

Essa função mede a largura em microssegundos de um pulso em qualquer pino digital. O parâmetro 'valor' diz à função que tipo de pulso deve ser medido, se HIGH ou LOW. O parâmetro 'espera' (time out) é opcional e se passado à função faz com que a medida do pulso só comece após o tempo em microssegundos ali especificado.

```
void loop() {

   pulseIn(4,HIGH);
   /*Aqui essa função vai monitorar o pino 4, e quando o nível nesse pino
   mudar de LOW para HIGH a sua largura vai ser medida até que seu nível
   volte para LOW. Se, por exemplo, for passado o valor 100 como terceiro
   parâmetro, a medida da largura do pulso só será disparada após 100 us*/

}
```

2.2 – Os pino analógicos

Os pinos analógicos são 6 pinos em uma só barra com o nome **ANALOG IN**, localizado no lado oposto às barras dos pinos digitais. São numerados de 0 a 5, agora da esquerda para a direita. Esses pinos são usados para leitura de sinais analógicos de sensores conectados ao Arduino, e podem ser de quaisquer valores entre zero e 5 volts. Os pinos de entradas analógicas não precisam ser previamente configurados com a função **pinMode( ).**



analogRead(pino)

Essa função lê o nível analógico presente no pino indicado pelo parâmetro entre parênteses e, após a conversão para o seu equivalente em bits, o guarda em uma variável determinada pelo programador.

```
void loop() {

  int sensor = analogRead(A0);
  /* Aqui a variável inteira 'sensor' vai armazenar a tensão
  analógica convertida para digital presente no pino A0.
  Essa informação vai ser um valor inteiro entre 0 (para 0 volt no pino)
  e 1023 (se 5 volts no pino). Uma tensão de  2,5 volts no pino A0 vai
  fazer a variável 'sensor' guardar o valor inteiro 512 */

}
```

** Os pinos analógicos são reconhecidos pela linguagem C do Arduino tanto como A0 a A5 como 14 a 19. Assim, a mesma expressão acima pode ser escrita também da seguinte forma:* ***int sensor = analogRead(14);***

** Uma observação importante em relação a esses pinos analógicos é que eles podem ser configurados também como pinos digitais pela função **pinMode(),** aumentando assim o número desse pinos para 20. Assim, a expressão **pinMode(14,OUTPUT);** transforma o pino analógico A0 em pino de saída digital como qualquer outro presente nas duas barras de pinos digitais.

```
/*
 * O código a seguir lê uma tensão CC no pino analógico A5 (pino 19)
 * e envia esse valor digitalizado para controlar o brilho de um
 * led no pino PWM 10;
*/

void setup() {
   // essa função pode ficar vazia se não configuração
}

void loop() {

  analogWrite(10,analogRead(A5)/4);
  // conversão de 0-1023 para 0-255 pela divisão por 4.

}
```

2.3 – A porta serial do Arduino e suas funções em C

Funções

Serial.begin(taxa)

Essa função habilita a porta serial e fixa a **taxa** de transmissão e recepção em bits por segundo entre o computador e o Arduino.

```
void setup() {
    Serial.begin(9600);
    /* Nesse exemplo essa função fixa a taxa de comunicação em 9600 bps.
    Os pinos digitais 0 e 1 não podem ser utilizados como entrada ou
    como saída de dados quando a porta serial e habilitada por essa
    função */
}
```

** Essa função vai sempre dentro da função **setup( ).***

Serial.end( )

Desabilita a porta serial para permitir o uso dos pinos digitais 0 e 1 para entrada e saída de dados.

** Essa função também deverá ser escrita dentro da função **setup( ).***

Serial.available( )

A função **Serial.available( )** retorna o número de bytes disponíveis para leitura no buffer da porta serial.

```
void setup() {
    int total = Serial.available();
    /* Aqui a variável inteira 'total' vai guardar o número de caracteres
    que estão disponíveis para leitura na porta serial */
}
```

** O valor 0 é retornado quando não há nenhuma informação para ser resgatada na porta serial.*

Serial.read( )

A função **Serial.read()** lê o primeiro byte que está no buffer da porta serial.

```
void setup() {
    int total = Serial.read();
    /* Aqui a variável inteira 'valor' vai guardar o primeiro byte (caracter)
    disponível na porta serial */
}
```

** O valor -1 é retornado quando não há nenhuma informação para ser resgatada na porta serial.*

Serial.print(valor, formato)

Essa função envia para a porta serial um caractere ASCII, que pode ser capturado por um terminal de comunicação. O segundo parâmetro, **'formato'**, é opcional e especifica com quantas casas decimais ou com que base numérica vai ser o número transmitido.

```
void loop() {

 Serial.print(1.23456);                // transmite 1.23 (default)
 Serial.print(1.23456, 3);             // transmite 1.234
 Serial.print("Jornada Automacao");    // transmite a frase (string)
 Serial.print('A');                    // transmite o caracter A
 Serial.print('A',BIN);                // transmite 01000001
 Serial.print('A',OCT);                // transmite o octal 101
 Serial.print('A',HEX);                // transmite o hexa 41
 Serial.print('A',DEC);                // transmite o decimal 65

}
```

Serial.println(valor, formato)

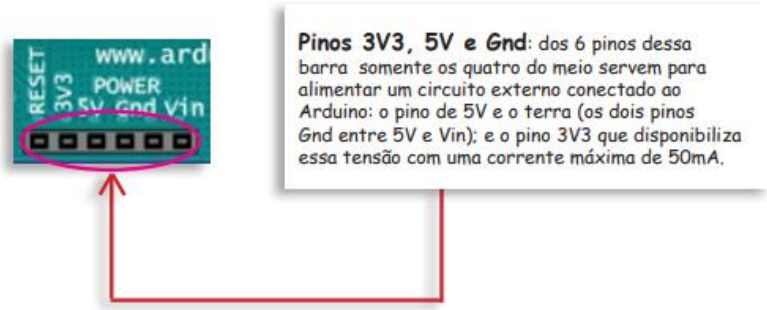
Como a anterior essa função enviar para a porta serial um caracter ASCII com os mesmos parâmetros opcionais de **'formato'**, porem acrescentando ao final da transmissão o caracter **Carriage Return** (retorno ao início da linha) e o caracter **New Line** (mudança para a próxima linha).

```
/*
 * Todas essas funções em C para a comunicação serial podem ser testadas com
 * o código abaixo e ativando o Terminal de Comunicação do Arduino
*/

void setup() {
    Serial.begin(9600);              // inicial a porta serial em 9600 bps
}

void loop() {

 Serial.print("Retorno de 'available()' : "); // envia frase ao terminal
 Serial.println(Serial.available());          // transmite total de bytes disponível
 delay(1000);                                 // aguarda 1 segundo
 Serial.print("Retorno de 'read()' : ");      // envia frase ao terminal
 Serial.println(Serial.read());               // transmite primeiro byte disponível
 delay(1000);                                 // aguarda 1 segundo

}
```

** O Terminal Serial do Arduino mostrará inicialmente todo segundo o retorno da função **available()**, que será 0, e da função **read()**, que será -1. Isso ocorre porque nenhum dado está disponível na porta serial do computador. Entre no Terminal do Arduino e transmita, por exemplo, as letras **ABCDE**, juntas (digite no campo ao lado do botão 'Enviar') e observe que a função **available()** informa inicialmente que existem 5 caracteres no buffer para ser enviados; e a função **read()** mostra o código ASCII decimal 65 do primeiro deles, que corresponde a letra **A.** Os outros caracteres vão sendo enviados sequencialmente enquanto **available()** vai decrementando até 0 de novo.*

2.4 – Os pinos de alimentação

Os pinos de alimentação ficam na barra com 6 pinos, marcada como **POWER**, localizada ao lado dos pinos analógicos. O primeiro pino dessa barra, **RESET**, quando forçado ao potencial de terra serve para resetar o Arduino. Do outro lado **VIN** é um pino que também pode servir para alimentar o Arduino se nele for aplicado uma tensão entre 7 e 12volts.



Este documento é complementar as vídeo aulas de programação básica do curso Automação na Veia.

Fontes :

“revistadoarduino”

“arduino.cc”